

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3 & 4**



**SINGLE LINKED LIST CIRCULAR DAN DOUBLE LINKED
LIST CIRCULAR**

Oleh:

Muhammad Rifqi Rahman

NIM. 2510817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
TAHUN 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3 & 4

Laporan Praktikum Algoritma & Struktur Data Modul 3 & 4: Single Linked List Circular dan Double Linked List Circular ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rifqi Rahman
NIM : 2510817310003

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1.....	1
A. Source Code.....	1
B. Pembahasan	7
SOAL 2.....	9
A. Source Code.....	10
B. Output Program	12
C. Pembahasan	12
SOAL 3.....	14
A. Source Code.....	14
B. Pembahasan	20
SOAL 4.....	22
A. Source Code.....	23
B. Output Program	27
C. Pembahasan	27
TAUTAN GIT	29

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2	12
Gambar 2. Screenshot Hasil Jawaban Soal 4	27

DAFTAR TABEL

Tabel 1 Source Code single_linked_list.cpp	1
Tabel 2 Source Code prob_a.cpp.....	10
Tabel 3 Source Code double_linked_list.cpp	14
Tabel 4 Source Code prob_b.cpp	23

SOAL 1

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new/delete)

Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 1 Source Code single_linked_list.cpp

1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>#include <stdexcept></code>
4	
5	<code>void SingleLinkedList::init() {</code>
6	<code> head = nullptr;</code>
7	<code> tail = nullptr;</code>
8	<code> size = 0;</code>
9	<code>}</code>
10	
11	<code>bool SingleLinkedList::is_empty() {</code>
12	<code> return size == 0;</code>
13	<code>}</code>
14	

```

15 void SingleLinkedList::add_front(int val) {
16     Node* newNode = new Node{val, nullptr};
17     if (is_empty()) {
18         head = tail = newNode;
19         tail->next = head;
20     } else {
21         newNode->next = head;
22         head = newNode;
23         tail->next = head;
24     }
25     size++;
26 }
27
28 void SingleLinkedList::add_back(int val) {
29     Node* newNode = new Node{val, nullptr};
30     if (is_empty()) {
31         head = tail = newNode;
32         tail->next = head;
33     } else {
34         tail->next = newNode;
35         tail = newNode;
36         tail->next = head;
37     }
38     size++;
39 }
40
41 void SingleLinkedList::add_idx(int val, int idx)
42 {
43     if (idx < 0 || idx > size) {
44         throw std::out_of_range("Index out of
45         bounds");

```

```

44     }
45
46     if (idx == 0) {
47         add_front(val);
48         return;
49     }
50     if (idx == size) {
51         add_back(val);
52         return;
53     }
54
55     Node* newNode = new Node{val, nullptr};
56     Node* curr = head;
57
58     for (int i = 0; i < idx - 1; ++i) {
59         curr = curr->next;
60     }
61
62     newNode->next = curr->next;
63     curr->next = newNode;
64     size++;
65 }
66
67 void SingleLinkedList::delete_front() {
68     if (is_empty()) {
69         throw std::out_of_range("List is
empty");
70     }
71
72     Node* temp = head;
73     if (size == 1) {

```



```

74         head = tail = nullptr;
75     } else {
76         head = head->next;
77         tail->next = head;
78     }
79     delete temp;
80     size--;
81 }
82
83 void SingleLinkedList::delete_back() {
84     if (is_empty()) {
85         throw std::out_of_range("List is
empty");
86     }
87
88     if (size == 1) {
89         delete_front();
90         return;
91     }
92
93     Node* curr = head;
94     while (curr->next != tail) {
95         curr = curr->next;
96     }
97
98     Node* temp = tail;
99     tail = curr;
100    tail->next = head;
101    delete temp;
102    size--;
103 }

```

```

104
105 void SingleLinkedList::delete_idx(int idx) {
106     if (idx < 0 || idx >= size) {
107         throw std::out_of_range("Index out of
bounds");
108     }
109
110     if (idx == 0) {
111         delete_front();
112         return;
113     }
114     if (idx == size - 1) {
115         delete_back();
116         return;
117     }
118
119     Node* curr = head;
120     for (int i = 0; i < idx - 1; ++i) {
121         curr = curr->next;
122     }
123
124     Node* temp = curr->next;
125     curr->next = temp->next;
126     delete temp;
127     size--;
128 }
129
130 void SingleLinkedList::display() {
131     if (is_empty()) {
132         std::cout << "List is empty\n";
133         return;

```

```

134     }
135
136     Node* curr = head;
137     for (int i = 0; i < size; ++i) {
138         std::cout << curr->data;
139         if (i < size - 1) std::cout << " ";
140         curr = curr->next;
141     }
142     std::cout << "\n";
143 }
144
145 int SingleLinkedList::get(int idx) {
146     if (idx < 0 || idx >= size) {
147         throw std::out_of_range("Index out of
148         bounds");
149     }
150     Node* curr = head;
151     for (int i = 0; i < idx; ++i) {
152         curr = curr->next;
153     }
154     return curr->data;
155 }
156
157 void SingleLinkedList::set(int val, int idx) {
158     if (idx < 0 || idx >= size) {
159         throw std::out_of_range("Index out of
160         bounds");
161     }
162     Node* curr = head;

```

```

163     for (int i = 0; i < idx; ++i) {
164         curr = curr->next;
165     }
166     curr->data = val;
167 }
168
169 void SingleLinkedList::clear() {
170     while (!is_empty()) {
171         delete_front();
172     }
173 }

```

B. Pembahasan

Pada baris [1, 2, dan 3] syntax *#include* mengimpor file header lokal dan library standar C++ dengan kompleksitas waktu dan ruang $O(1)$, pada baris [5 hingga 8] syntax *init()* menyetel nilai pointer memori menjadi *nullptr* untuk inisialisasi awal, pada baris [11 dan 12] syntax *is_empty()* mengevaluasi sisa kapasitas elemen list, pada baris [15 dan 28] syntax *add_front()* dan *add_back()* mengalokasikan ruang memori dinamis secara manual di dalam heap menggunakan perintah *new Node*, pada baris [19 dan 36] syntax *tail->next = head* merajut pointer menyilang agar struktur melingkar secara otomatis dengan kompleksitas $O(1)$, lalu pada baris [41, 58, dan 59] syntax *add_idx()* beserta perulangan *for* menelusuri posisi indeks memori secara linier yang membuat algoritma memakan kompleksitas waktu $O(n)$ dengan ruang $O(1)$.

Pada baris [67] syntax *delete_front()* memperbarui pergeseran rujukan awal pointer node, pada baris [79, 100, dan 126] syntax *delete* melepaskan alokasi memori manual kembali ke sistem operasi, pada baris [83 dan 105] syntax *delete_back()* dan *delete_idx()* melakukan proses pencabutan node iteratif berskala waktu $O(n)$ dan ruang $O(1)$, pada baris [130, 145, dan 157] syntax *display()*, *get()*, dan *set()* mengakses serta memanipulasi rentetan data memori, pada baris [137, 151, dan 163] syntax *for* yang dibatasi kapasitas *size* mencegah sistem terjebak siklus melingkar abadi (infinite loop)

dengan proses penelusuran linier bernilai $O(n)$, lalu pada baris [169 hingga 171] syntax *clear()* dan *while (!is_empty())* memanggil perintah *delete_front()* secara berkelanjutan yang menjamin algoritma ini sangat valid dalam menghancurkan total seluruh objek *new* secara presisi $O(n)$ demi menghindari kebocoran memori (memory leak).

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 104$
- $1 \leq K \leq 109$
- $1 \leq A_i \leq 106$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K

A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

A. Source Code

Tabel 2 Source Code prob_a.cpp

```
1  #include <iostream>
2  #include "single_linked_list.h"
3
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n;
11     int k;
12
13     if (!(cin >> n >> k)) return 0;
14
15     SingleLinkedList list;
16     list.init();
17
18     for (int i = 0; i < n; i++) {
19         int val;
20         cin >> val;
21         list.add_back(val);
22     }
23 }
```

```

24     int initial_k = k;
25     int current_k = k;
26
27     Node* curr = list.head;
28     Node* prev = list.tail;
29
30     while (list.size > 1) {
31         int steps = (current_k - 1) % list.size;
32
33         for (int i = 0; i < steps; i++) {
34             prev = curr;
35             curr = curr->next;
36         }
37
38         int destroyed_val = curr->data;
39
40         prev->next = curr->next;
41         if (curr == list.head) list.head = curr-
>next;
42         if (curr == list.tail) list.tail = prev;
43
44         Node* target = curr;
45         curr = curr->next;
46         delete target;
47         list.size--;
48
49         if (destroyed_val % 2 == 0) {
50             current_k++;
51         } else {
52             current_k--;
53         }

```

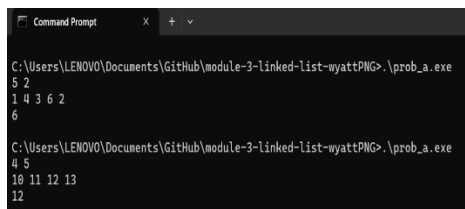


```

54
55         if (current_k <= 0) {
56             current_k = initial_k;
57         }
58     }
59
60     if (list.size == 1) {
61         cout << list.head->data << "\n";
62     }
63
64     list.clear();
65
66     return 0;
67 }

```

B. Output Program



```

Command Prompt
C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyatt\prob_a.exe
5 2
1 4 3 6 2
6
C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyatt\prob_a.exe
4 5
10 11 12 13
12

```

Gambar 1. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1 dan 2] syntax `#include <iostream>` dan `#include "single_linked_list.h"` mengimpor library standar C++ serta file header lokal yang memuat struktur data dengan kompleksitas waktu dan ruang $O(1)$, pada baris [4] syntax `using namespace std;` mendeklarasikan penggunaan ruang nama standar, pada baris [6] syntax `int main()` menginisiasi blok fungsi utama eksekusi program, pada baris [7 dan 8] syntax `ios_base::sync_with_stdio(false);` dan `cin.tie(NULL);` mematikan sinkronisasi I/O bawaan demi mempercepat laju pembacaan data, pada baris [10 hingga 13] syntax `cin >> n >> k` digunakan untuk membaca nilai jumlah elemen dan langkah awal sekaligus memvalidasi masukannya, pada baris [15 dan 16] syntax

list.init() memanggil inisialisasi awal senarai melingkar yang kosong, pada baris [18 hingga 22] syntax *list.add_back(val)* di dalam perulangan *for* digunakan untuk memesan dan mengalokasikan ruang memori secara manual ke dalam heap untuk tiap elemen baru sehingga memakan kompleksitas waktu $O(N)$ dan ruang $O(N)$, pada baris [24 hingga 28] syntax *Node* curr* dan *Node* prev* memposisikan titik awal penelusuran pointer pada struktur sirkuler, pada baris [30] syntax *while (list.size > 1)* menciptakan perulangan eliminasi utama yang secara keseluruhan membentuk algoritma berjalan kompleksitas waktu $O(N^2)$ karena adanya penelusuran berulang, pada baris [31] syntax *(current_k - 1) % list.size* adalah bentuk optimasi logis krusial yang digunakan untuk memangkas iterasi pencarian berlebih menggunakan operasi modulo yang berjalan secepat $O(1)$, pada baris [33 hingga 36] syntax *for* menggeser pointer penelusuran langkah demi langkah untuk menemukan target eliminasi, pada baris [39 hingga 42] syntax *prev->next = curr->next* melakukan manuver manipulasi pointer $O(1)$ untuk melepaskan target dari rantai senarai tanpa memutus sifat melingkarnya, pada baris [44 hingga 47] syntax *delete target;* digunakan secara tegas untuk menghancurkan node dari area *heap* dan melepaskan memorinya secara manual yang membuktikan validitas algoritma ini dalam menghindari ancaman *memory leak*, pada baris [49 hingga 57] syntax percabangan *if* memodifikasi arah dan sisa energi lompatan *current_k* dengan mengevaluasi kondisi genap atau ganjil secara $O(1)$, pada baris [60 dan 61] syntax *cout << list.head->data* digunakan untuk mencetak satu-satunya node terakhir yang selamat ke layar, pada baris [64] syntax *list.clear()* memanggil fungsi pelucutan memori sapu bersih yang menjamin program ini tuntas dan terbebas sepenuhnya dari kebocoran memori dengan menghancurkan seluruh sisa alokasi objek secara manual, lalu terakhir pada baris [66] syntax *return 0;* digunakan untuk mengakhiri fungsi utama dengan kode keberhasilan ke sistem operasi.

SOAL 3

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new/delete)

Constraints: Strictly NO STL, NO Global Arrays.

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 3 Source Code double_linked_list.cpp

```
1  #include "double_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void DoubleLinkedList::init() {
6      head = nullptr;
7      tail = nullptr;
8      size = 0;
9  }
10
11 bool DoubleLinkedList::is_empty() {
12     return size == 0;
13 }
```

```

14
15 void DoubleLinkedList::add_front(int val) {
16     Node*      newNode      =      new
Node{static_cast<char>(val), nullptr, nullptr};
17
18     if (is_empty()) {
19         head = tail = newNode;
20         head->next = head;
21         head->prev = head;
22     } else {
23         newNode->next = head;
24         newNode->prev = tail;
25         head->prev = newNode;
26         tail->next = newNode;
27         head = newNode;
28     }
29     size++;
30 }
31
32 void DoubleLinkedList::add_back(int val) {
33     Node*      newNode      =      new
Node{static_cast<char>(val), nullptr, nullptr};
34
35     if (is_empty()) {
36         head = tail = newNode;
37         head->next = head;
38         head->prev = head;
39     } else {
40         newNode->next = head;
41         newNode->prev = tail;
42         tail->next = newNode;

```

```

43         head->prev = newNode;
44         tail = newNode;
45     }
46     size++;
47 }
48
49 void DoubleLinkedList::add_idx(int val, int idx)
50 {
51     if (idx < 0 || idx > size) {
52         throw std::out_of_range("Index out of
53 bounds");
54     }
55
56     if (idx == 0) {
57         add_front(val);
58         return;
59     }
60
61     if (idx == size) {
62         add_back(val);
63         return;
64     }
65
66     Node* curr = head;
67     for (int i = 0; i < idx; ++i) {
68         curr = curr->next;
69     }
70
71     Node* newNode = new
Node{static_cast<char>(val), curr, curr->prev};
72     curr->prev->next = newNode;
73     curr->prev = newNode;

```

```

71         size++;
72     }
73
74     void DoubleLinkedList::delete_front() {
75         if (is_empty()) {
76             throw std::out_of_range("Index out of
77             bounds");
78         }
79         Node* temp = head;
80         if (size == 1) {
81             head = tail = nullptr;
82         } else {
83             head = head->next;
84             head->prev = tail;
85             tail->next = head;
86         }
87         delete temp;
88         size--;
89     }
90
91     void DoubleLinkedList::delete_back() {
92         if (is_empty()) {
93             throw std::out_of_range("Index out of
94             bounds");
95         }
96         Node* temp = tail;
97         if (size == 1) {
98             head = tail = nullptr;
99         } else {

```

```

100         tail = tail->prev;
101         tail->next = head;
102         head->prev = tail;
103     }
104     delete temp;
105     size--;
106 }
107
108 void DoubleLinkedList::delete_idx(int idx) {
109     if (idx < 0 || idx >= size) {
110         throw std::out_of_range("Index out of
111         bounds");
112     }
113     if (idx == 0) {
114         delete_front();
115         return;
116     }
117     if (idx == size - 1) {
118         delete_back();
119         return;
120     }
121
122     Node* curr = head;
123     for (int i = 0; i < idx; ++i) {
124         curr = curr->next;
125     }
126
127     curr->prev->next = curr->next;
128     curr->next->prev = curr->prev;
129     delete curr;

```

```

130         size--;
131     }
132
133 void DoubleLinkedList::display() {
134     if (is_empty()) {
135         std::cout << "List is empty\n";
136         return;
137     }
138
139     Node* curr = head;
140     for (int i = 0; i < size; ++i) {
141         std::cout << curr->data;
142         if (i < size - 1) std::cout << " ";
143         curr = curr->next;
144     }
145     std::cout << "\n";
146 }
147
148 char DoubleLinkedList::get(int idx) {
149     if (idx < 0 || idx >= size) {
150         throw std::out_of_range("Index out of
151         bounds");
152     }
153
154     Node* curr = head;
155     for (int i = 0; i < idx; ++i) {
156         curr = curr->next;
157     }
158     return curr->data;
159 }

```



```

160 void DoubleLinkedList::set(char val, int idx) {
161     if (idx < 0 || idx >= size) {
162         throw std::out_of_range("Index out of
163         bounds");
164     }
165     Node* curr = head;
166     for (int i = 0; i < idx; ++i) {
167         curr = curr->next;
168     }
169     curr->data = val;
170 }
171
172 void DoubleLinkedList::clear() {
173     while (!is_empty()) {
174         delete_front();
175     }
176 }

```

B. Pembahasan

Pada baris [1 hingga 3] syntax *#include* mengimpor file header lokal dan library standar C++ untuk penanganan eksepsi dan input-output dengan kompleksitas waktu dan ruang $O(1)$, pada baris [5 hingga 9] syntax *init()* menyetel nilai pointer memori menjadi *nullptr* serta *size* menjadi 0 untuk inisialisasi awal, pada baris [11 dan 12] syntax *is_empty()* mengevaluasi sisa kapasitas elemen list, pada baris [15 dan 32] syntax *add_front()* dan *add_back()* mengalokasikan ruang memori dinamis secara manual di dalam heap menggunakan perintah *new Node*, pada baris [20, 21, 37, dan 38] syntax *head->next = head* dan *head->prev = head* beserta modifikasi pointer di dalam blok *else* merajut pointer menyilang secara dua arah agar struktur melingkar ganda beroperasi otomatis dengan kompleksitas $O(1)$, lalu pada baris [49, 64, dan 65] syntax *add_idx()* beserta perulangan *for* menelusuri posisi indeks memori secara linier yang

membuat algoritma penyisipan ini memakan kompleksitas waktu $O(n)$ dengan penggunaan ruang $O(1)$.

Pada baris [74 dan 91] syntax *delete_front()* dan *delete_back()* memperbarui pergeseran rujukan awal dan akhir pointer node secara instan senilai $O(1)$, pada baris [87, 104, dan 129] syntax *delete* bertugas esensial melepaskan alokasi memori manual kembali ke sistem operasi dari area heap, pada baris [108, 124, dan 125] syntax *delete_idx()* menelusuri rantai memori menggunakan perulangan untuk melakukan pemutusan pointer node berskala waktu $O(n)$ dan ruang $O(1)$, pada baris [133, 148, dan 160] syntax *display()*, *get()*, dan *set()* mengakses serta memanipulasi rentetan karakter data memori, pada baris [140, 154, dan 166] syntax *for* yang secara ketat dibatasi oleh nilai properti *size* digunakan untuk mencegah sistem terjebak di dalam siklus melingkar abadi (infinite loop) melalui proses penelusuran linier bernilai $O(n)$, lalu pada baris [172 hingga 175] syntax *clear()* dan *while (!is_empty())* memanggil perintah *delete_front()* secara berkelanjutan yang mengonfirmasi bahwa algoritma ini sangat valid dan diyakini benar dalam menghancurkan total seluruh objek bentukan *new* secara terstruktur dengan kompleksitas $O(n)$ demi menghindari bencana kebocoran memori (memory leak).

SOAL 4

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

- Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
- Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
- Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 1018$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C_1 C_2 ... C_N

X_1 Y_1

X2 Y2

...

XR YR

Keterangan: *C_i* adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

Contoh Kasus

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

A. Source Code

Tabel 4 Source Code prob_b.cpp

1	#include <iostream>
2	#include <utility>
3	#include "double_linked_list.h"
4	
5	using namespace std;
6	
7	int main() {
8	ios_base::sync_with_stdio(false);
9	cin.tie(NULL);
10	

```

11     int n, r;
12     if (!(cin >> n >> r)) return 0;
13
14     DoubleLinkedList list;
15     list.init();
16
17     for (int i = 0; i < n; i++) {
18         char c;
19         cin >> c;
20         list.add_back(c);
21     }
22
23     if (list.size == 0) {
24         cout << "EMPTY\n";
25         return 0;
26     }
27
28     Node* alpha = list.head;
29     Node* beta = list.tail;
30
31     for (int i = 0; i < r; i++) {
32         long long x, y;
33         cin >> x >> y;
34
35         if (list.size == 0) continue;
36
37         long long moves_alpha = x % list.size;
38         if (moves_alpha > list.size / 2) {
39             long long back_moves = list.size -
moves_alpha;

```

```

40         for (long long j = 0; j < back_moves;
j++) alpha = alpha->prev;
41     } else {
42         for (long long j = 0; j < moves_alpha;
j++) alpha = alpha->next;
43     }
44
45     long long moves_beta = y % list.size;
46     if (moves_beta > list.size / 2) {
47         long long forward_moves = list.size
- moves_beta;
48         for (long long j = 0; j <
forward_moves; j++) beta = beta->next;
49     } else {
50         for (long long j = 0; j < moves_beta;
j++) beta = beta->prev;
51     }
52
53     if (alpha == beta) {
54         Node* target = alpha;
55
56         if (list.size == 1) {
57             list.head = nullptr;
58             list.tail = nullptr;
59             alpha = nullptr;
60             beta = nullptr;
61         } else {
62             Node* next_a = target->next;
63             Node* next_b = target->prev;
64

```

```

65         target->prev->next = target-
>next;
66         target->next->prev = target-
>prev;
67
68         if (target == list.head) list.head
= target->next;
69         if (target == list.tail) list.tail
= target->prev;
70
71         alpha = next_a;
72         beta = next_b;
73     }
74
75     delete target;
76     list.size--;
77
78     } else {
79         bool is_adjacent = false;
80
81         if (alpha->next == beta && alpha !=
list.tail) is_adjacent = true;
82         if (beta->next == alpha && beta !=
list.tail) is_adjacent = true;
83
84         if (is_adjacent) {
85             swap(alpha->data, beta->data);
86         }
87     }
88 }
89

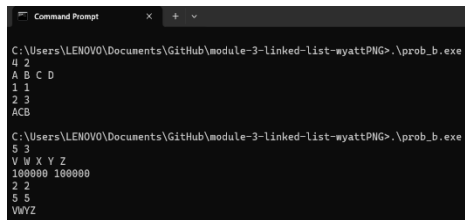
```

```

90     if (list.size == 0) {
91         cout << "EMPTY\n";
92     } else {
93         Node* curr = list.head;
94         for (int i = 0; i < list.size; i++) {
95             cout << curr->data;
96             curr = curr->next;
97         }
98         cout << "\n";
99     }
100
101     list.clear();
102
103     return 0;
104 }

```

B. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyattPNG>.\prob_b.exe
4 2
A B C D
1 1
ACB

C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyattPNG>.\prob_b.exe
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ

```

Gambar 2. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [1 hingga 3] syntax `#include` mengimpor library standar C++ dan file header lokal pembentuk struktur senarai dengan kompleksitas waktu dan ruang $O(1)$, pada baris [7 hingga 12] syntax `int main()` menginisiasi program serta membaca variabel masukan simulasi dengan I/O yang dioptimasi, pada baris [14 hingga 21] syntax `list.add_back(c)` digunakan di dalam perulangan untuk mengalokasikan ruang memori dinamis secara manual ke dalam area heap bagi setiap elemen masukan guna membangun senarai berantai dengan kompleksitas waktu dan ruang $O(N)$, pada baris [28 dan 29] syntax `Node* alpha` dan `Node* beta` menempatkan posisi awal penelusuran

proyektor, pada baris [31] syntax *for* menciptakan siklus utama simulasi ronde berjalan kompleksitas waktu $O(R)$, pada baris [37 dan 45] syntax *% list.size* adalah optimasi matematis krusial untuk memangkas pergerakan melingkar berlebih menggunakan operasi modulo yang dieksekusi konstan secepat $O(1)$, pada baris [38 hingga 51] syntax penelusuran pointer *prev* atau *next* menentukan rute langkah terpendek pergerakan secara proporsional bernilai waktu $O(N)$ dan ruang $O(1)$, pada baris [53] syntax *if (alpha == beta)* mengevaluasi kondisi tabrakan singular secara instan $O(1)$, pada baris [62 hingga 72] syntax modifikasi pointer seperti *target->prev->next = target->next* bermanuver memisahkan target dari rantai secara $O(1)$ tanpa merusak integrasi strukturnya, pada baris [75] syntax *delete target;* bertugas esensial menghancurkan node tersebut secara manual dari heap untuk membebaskan ruang memori, pada baris [78 hingga 87] syntax blok *else* mendeteksi resonansi bersebelahan dengan syarat spesifik *alpha != list.tail* guna menukar sifat data menggunakan fungsi *swap* berkecepatan $O(1)$ tanpa menyalahi batasan ujung dimensi linier, pada baris [90 hingga 99] syntax *cout* dan perulangan *for* mencetak karakter yang tersisa beruntun dari *head* berbekal kompleksitas $O(N)$, lalu pada baris [101] syntax *list.clear()* memusnahkan keseluruhan sisa memori node secara sapu bersih yang menjamin kemutlakan validitas algoritma ini dalam menghindari ancaman kebocoran memori (memory leak) sebelum program diakhiri dengan syntax *return 0;* pada baris [103].

TAUTAN GIT

<https://github.com/DSA26-ULM/module-3-linked-list-wyattPNG>